ФАКУЛТЕТ ЗА ИНФОРМАТИЧКИ НАУКИ И КОМПЈУТЕРСКО ИНЖЕНЕРСТВО

Техники за креирање алгоритми

Алгоритми и податочни структури

Аудиториска вежба 3

Техника на груба сила (brute force)

- Техниката на груба сила го решава проблемот на наједноставниот, најдиректниот или најочигледниот начин
- Како резултат, на ваков тип на алгоритам му треба многу повеќе работа за да реши одреден проблем од некој попаметен или пософистициран алгоритам
- Од друга страна, алгоритмите базирани на груба сила често се поедноставни за имплементација отколку пософистицираните
- Се разгледуваат сите можности

Задача 1

- Дадени се N точки (N >= 2) во дводимензионален простор. Пресметајте кое е најмалото растојание помеѓу две точки.

Задача 1 - Java

```
class Point {
    double x;
    double y;

    Point(double x, double y) {
        this.x = x;
        this.y = y;
    }
}

class BruteForce {
    int INF = 1000000;

    double min(double a, double b) {
        if (a < b) {
            return a;
        } else {
            return b;
        }
    }
```

Задача 1 - Java

```
    double rastojanie(Point a, Point b) {
        return Math.sqrt((a.x - b.x) * (a.x - b.x) +
             (a.y - b.y) * (a.y - b.y));
    }

    double najmalo_rastojanie(Point p[], int n) {
        int i, j;
        double best = INF;

        for (i = 0; i < (n - 1); i++) {
            for (j = i + 1; j < n; j++) {
                best = Math.min(best, rastojanie(p[i], p[j]));
            }
        }

        return best;
    }
}
```

Задача 2

- Дадена е шаховска табла. Пресметајте на колку различни начини можат да се постават две “кралици” без да се напаѓаат една со друга. Две “кралици” се напаѓаат ако се наоѓаат во ист ред, колона или дијагонала.

Задача 2 - Java

```
class BruteForce {


    int dali_se_napagaat(int i1, int j1, int i2, int j2) {
        if (i1 == i2) // ista redica
            return 1;
        if (j1 == j2) // ista kolona
            return 1;
        if (Math.abs(i1 - i2) == Math.abs(j1 - j2)) // ista dijagonala
            return 1;
        return 0;
    }
```

Задача 2 - Java

```
    int broj_na_nacini() {
        int i1, j1, i2, j2;
        int rezultat = 0;

        for (i1 = 0; i1 < 8; i1++) {
            for (j1 = 0; j1 < 8; j1++) {
                for (i2 = 0; i2 < 8; i2++) {
                    for (j2 = 0; j2 < 8; j2++) {
                        if (dali_se_napagaat(i1, j1, i2, j2) == 0) {
                            rezultat++;
                        }
                    }
                }
            }
        }

        return rezultat;
    }
}
```

Алчни алгоритми

- Архетипот "Алчен алгоритам" ("Greedy algorithm") е алгоритам кој користи некоја хевристика за да најде оптимално решение на некој проблем.
- Проблемот се решава во фази, притоа земајќи во предвид само една влезна променлива во даден момент. Во секоја фаза се одлучува за локалниот оптимален избор, преку кое се стигнува до глобален оптимум.
- Алчните алгоритми најчесто се извршуваат многу побрзо од техниките на груба сила.
- Не секогаш алчната стратегија води до точното решение

Задача 3

- Дадени се парички со одредена вредност (постојат парички од 50, 10, 5, 2 и 1 денар), и притоа бројот на парички од секоја вредност е неограничен. За дадена сума да се определи најмалиот број на парички кои се потребни да се формира таа сума.

Задача 3 - решение

- При решението на овој проблем може да се примени архетипот „Алчен алгоритам“.
- Алчниот избор на парички се состои во земање на што повеќе парички од најголемата паричка (локален оптимум) за да се формира збир што поблизок до дадената сума. Овој избор на парички во моментот изгледа како најдобар и бројот на тие најголеми парички се додава на решението.
- Понатаму, за остатокот од сумата односно за потпроблемот повторно се продолжува со следната по големина најголема паричка и повторно се земаат што повеќе парички од дадената паричка. Така се продолжува со паричките подредени според големината се додека не се формира бараната сума со избраните парички.

Задача 3 - Java

```
class Greedy {

    void sortiraj_paricki(int coins[], int n) {
        int i, j, tmp;

        for (i = 0; i < n; i++) {
            for (j = i + 1; j < n; j++) {
                if (coins[i] < coins[j]) {
                    tmp = coins[i];
                    coins[i] = coins[j];
                    coins[j] = tmp;
                }
            }
        }
    }
```

Задача 3 - Java

```
// coins e niza so vrednostite na parickite koi se dadeni
// n e brojot na paricki
// sum e dadenata suma
// coinsNum e niza za resenieto so brojot na paricki od sekoja
// golemina na paricka
int greedyCoins(int coins[], int n, int sum, int coinsNum[]) {
    sortiraj_paricki(coins, n);

    int i = 0;
    int br = 0; // vkupniot broj na paricki za da se formira
        // dadenata suma

    while (sum > 0) {
        coinsNum[i] = sum / coins[i];
        sum -= coinsNum[i] * coins[i];
        br += coinsNum[i];
        i++;
    }
```

Задача 3 - Java

```
        while (i < n) {
            coinsNum[i] = 0;
            i++;
        }

        return br;
    }
}
```

Задача 3 - дискусија

- Дали овој алгоритам секогаш ќе даде оптимално решение? Колкав е минималниот број на монети за да се добие сума 13, а на располагање имаме монети од 1, 2, 5, 8, 10 денари?

Проблем на пакување на ранец (Knapsack problem)

- Математичка формулација
  - Нека w_i е тежина на *i*-тиот објект
  - Нека p_i е остварениот профит кога е земен *i*-тиот објект
  - Нека C е капацитетот на ранецот
  - Нека x_i е променлива чија вредност е
    - 0 или 1 за “0/1 knapsack problem”. $x_i = 1$ значи дека е земен *i*–тиот објект
    - $0 \leq x_i \leq 1$ за “fractional knapsack problem”. Може да се земе и дел од објектот

Проблем на пакување на ранец (Knapsack problem)

- Цел и ограничувања
  - При дадени $\{w_1, w_2, \ldots, w_n\}$ и $\{p_1, p_2, \ldots, p_n\}$
  - Цел: максимизирај $\sum_{i=1}^{n} p_i x_i$
  - Ограничување $\sum_{i=1}^{n} w_i x_i \leq C$

Задача 4

- Студент се спрема да оди на патување. Дозволено му е да носи еден куфер со максимална тежина од 20 кг. Студентот се двоуми помеѓу облека, книги и друштвени игри. Откако ги измерил и оценил овие работи, студентот ја добил следната табела:

#	Објект	Вредност	Тежина
1	Облека	200	10
2	Книги	150	20
3	Монопол, карти…	5	0.5
4	CD player, MP3 player…	80	5

Задача 4

- Што да понесе студентот, така што вредноста на понесените работи да биде максимална?
- Постојат две верзии на овој проблем:
  - **Fractional knapsack проблем**: Студентот може да земе делови од објекти односно може да се одлучи да понесе само еден дел x_i од објектот o_i, каде $0 \leq x_i \leq 1$.
  - **0-1 knapsack проблем**: проблемот е исто поставен, но објектите не можат да се поделат на помали делови, така што студентот ќе мора да се одлучи дали ќе го земе објектот или не. Значи не може да земе само дел од објектот ($x_i=0$ или $x_i=1$).

Задача 4 (fractional knapsack) – анализа

- За да се реши Fractional knapsack (предметите може да се земаат по делови) проблемот може да го користиме архетипот "Алчен алгоритам".
- На пример, нека е даден следниот проблем: бројот на објекти е 3 (n=3), капацитетот на ранецот е C=20, вредноста за секој од објектите е дадена со (p1, p2, p3) = (25, 24, 15) и тежините на објектите изнесуваат (t1, t2, t3) = (18, 15, 10). Да се пополни ранецот со објекти за да се добие максимален профит.

Задача 4 (fractional knapsack) – анализа

- Доколку можеме да земеме дел од дадените предмети без ограничувања, оптималното решение ќе се добие со пресметување на релативната вредност на предметот во однос на неговата тежина
- Алчен избор: прво земи ги предметите што вредат најмногу а тежат најмалку

Задача 4 (fractional knapsack) – анализа

- Се пресметува односот p/t, вредност според тежина
- Се подредуваат предметите според пресметаниот сооднос
- Ранецот прво се пополнува со предметите кои имаат најголема релативна вредност во однос на тежина

Задача 4 (fractional knapsack) -Java

```
class Greedy {

    void sortiraj(int p[], int t[], int n) {
        int i, j, tmpP, tmpT;

        for (i = 0; i < n; i++) {
            for (j = i + 1; j < n; j++) {
                if ((p[i]/(float)t[i])<(p[j]/(float) t[j])) {
                    tmpP = p[i];
                    tmpT = t[i];
                    p[i] = p[j];
                    t[i] = t[j];
                    p[j] = tmpP;
                    t[j] = tmpT;
                }
            }
        }

    }
```

Задача 4 (fractional knapsack) -Java

```
// p i t gi sodrzat profitot i tezinata na objektite
// C e kapacitet na paketot, x e vektor na resenieto
float grFractKnp(int p[], int t[], float C, int n, float x[]) {

    sortiraj(p, t, n);
    // objektite se podredeni taka da bide zadovolen
// p[i]/t[i] >= p[i+1]/t[i+1]

    int i;
    float profit = 0;

    for (i = 0; i < n; i++) {
        x[i] = 0;
    }
```

Задача 4 (fractional knapsack) -Java

```
    for (i = 0; i < n; i++) {
        if (C > t[i]) {          // objektot go stavame celosno
           x[i] = 1;
           C -= t[i];
            profit += p[i];
        } else {                 // objektot go stavame delumno
            x[i] = (C / (float) t[i]);
    profit += x[i] * (float) p[i];
            C = 0;
            break;
        }
    }

    return profit;
}
```

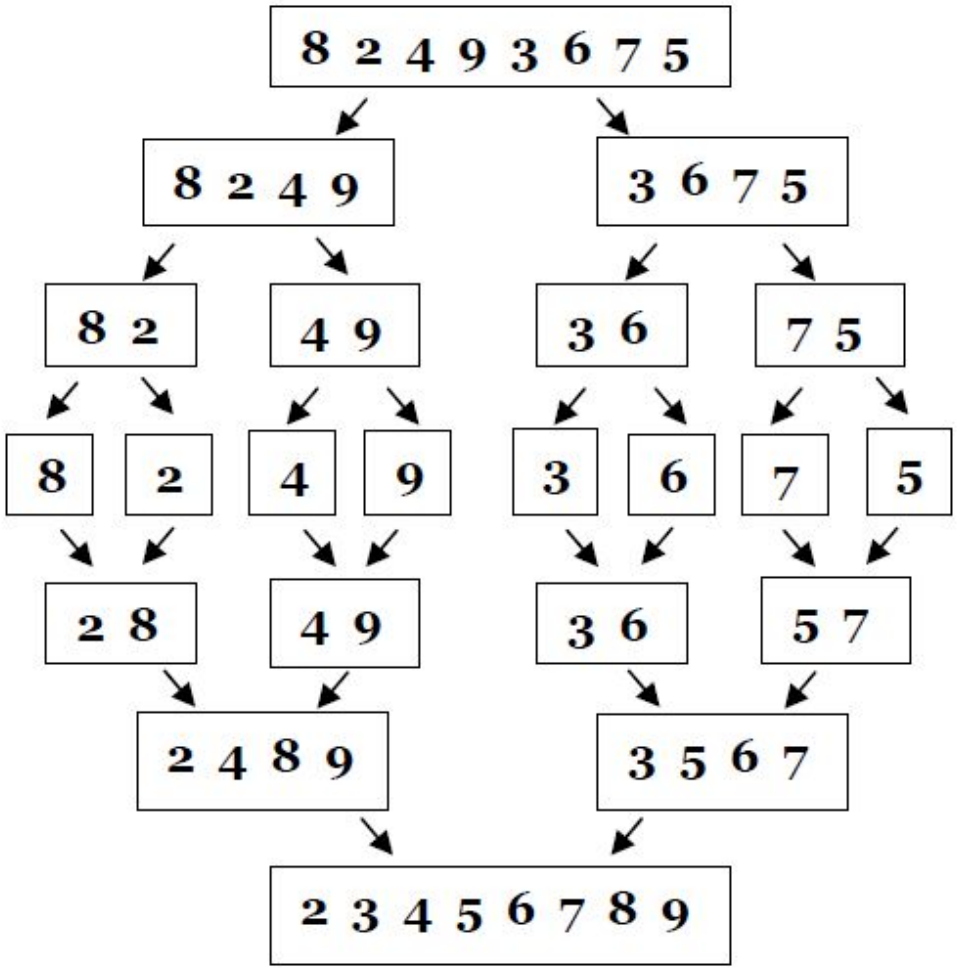
Раздели и владеј

- Чекорите на алгоритмот се следниве:
  - **Подели**: подели го проблемот на потпроблеми P_1, P_2, … , P_n се додека не се дојде до основен случај
  - **Владеј**: реши ги проблемите P_i (најчесто се користи рекурзивна примена на истиот алгоритам), добивајќи решенија R_i
  - **Слеј**: комбинирај ги решенијата R_i на потпроблемите за да го добиеш решението на почетниот проблем.

Сортирање со спојување (merge sort)

- "Раздели и владеј" алгоритамот во три чекори за mergesort сортирањето на некоја низа a[0 … n] е следниот:
  - **Подели**: низата a[0..n] се дели на половина на две поднизи
  - **Владеј**: секоја половина a[0 .. (n/2)] и a[(n/2) +1 .. n] се сортира рекурзивно
  - **Слеј**: двете сортирани половини се спојуваат во една целина

Сортирање со спојување (merge sort) - илустрација



Сортирање со спојување (merge sort)

```
class DivideAndConquer {
    //spojuvanje na dve sortirani nizi [l, mid], [mid+1, r]
    //rezultatot e nova sortirana niza
    void merge(int a[], int l, int mid, int r) {
        int numel = r - l + 1;
        int temp[] = new int[100]; // nova niza za privremeno cuvanje
                   // na sortiranite elementi
        int i = l, j = mid+1, k = 0;

        while ((i <= mid) && (j <= r)) {
            if (a[i] < a[j]) {
                temp[k] = a[i];
                i++;
            } else {
                temp[k] = a[j];
                j++;
            }
            k++;
        }
```

Сортирање со спојување (merge sort)

```
    while (i <= mid) {
        temp[k] = a[i];
        i++;
        k++;
    }

    while (j <= r) {
        temp[k] = a[j];
        j++;
        k++;
    }

    for (k = 0; k < numel; k++) {
        a[l + k] = temp[k];
    }
}
```

Сортирање со спојување (merge sort)

```
    void mergesort(int a[], int l, int r) {
        if (l == r) {
            return;
        }

        int mid = (l + r) / 2;
        mergesort(a, l, mid);
        mergesort(a, mid + 1, r);
        merge(a, l, mid, r);
    }
}
```

Сортирање со спојување (merge sort) - анализа

- Анализа за комплексноста на алгоритамот:

  $T(n) = 2T(n/2)+n$

  $2T(n/2) = 2(2(T(n/4))+n/2) = 4T(n/4)+n \rightarrow$
  $T(n) = 4T(n/4)+2n$

  $4T(n/4) = 4(2T(n/8))+(n/4) = 8T(n/8)+n \rightarrow$
  $T(n) = 8T(n/8)+3n$

  …

  $T(n) = 2^k T(n/ 2^k)+kn$ и ако замениме $k=\log(n)$ се добива: $T(n) = nT(1)+n \log(n) = n \log(n) + n$

  Значи комплексноста $O(n \log_2 n)$

Задача 5

- Да се напише функција која ќе го пресмета n-тиот степен на некој број со примена на техниката "Раздели и владеј".

Задача 5 - Java

```
class DivideAndConquer {

   int pow(int x, int n) {
        int r;

        if(n == 0)
            return(1);
        else if(n % 2 == 0) {
            r = pow(x, (n/2));
            return r*r;
        } else {
            r = pow(x, (n/2));
            return x*r*r;
        }
    }
}
```

Задача 5 - анализа

- Анализа за комплексноста на алгоритамот:

$T(n) = T(n/2) + O(1)$

$T(n/2) = (T(n/4) + 1) \rightarrow$

$T(n) = T(n/4) + 2$

$T(n/4) = T(n/8) + 1 \rightarrow$

$T(n) = T(n/8) + 3$

…

$T(n) = T(n/2^k)+k$ и ако замениме $k=\log(n)$ се добива: $T(n) = T(1)+\log(n) = \log(n)$

Значи комплексноста е *O(log n)*

Задача 5 - анализа

- Решение 2:

```
int pow(int x, int n) {
    int r;

    if(n == 0)
        return(1);
    else if(n % 2 == 0) {
        return pow(x, (n/2))*pow(x,
(n/2));
    } else {
        return x*pow(x, (n/2))*pow(x,
(n/2));
    }
}
```

- Што може да се заклучи?

Задача 5 (решение 2)- анализа

- Анализа за комплексноста на алгоритамот:

  $T(n) = 2T(n/2)+O(1)$

  $T(n/2) = 2(T(n/4) + 1) \rightarrow$

  $T(n) = 4T(n/4)+2$

  $T(n/4) = 8T(n/8) + 1 \rightarrow$

  $T(n) = 8T(n/8)+3$

  …

  $T(n) = 2^kT(n/2^k)+k$ и ако замениме $k=\log(n)$ се добива: $T(n) = nT(1)+\log(n) = n + \log(n)$

  Значи комплексноста е $O(n)$